

Name: Namratha Reddy
BU ID: U65966062
Course: CS-501 E1

A1 (Mobility Lens) Report

Package: com.namratha.mobilitylens

Emulator: Pixel 9

API level: API 24

SDK configuration: minSdk 24, targetSdk 37, compileSdk 37

Application and SDK Configuration:

Mobility Lens is my first single screen Kotlin and Jetpack Compose application. Even though the displayed dimension changes when Previous or Next is pressed, those buttons update the Compose state and recompose the same screen instead of navigating to a new Activity or screen. My simple application displays the six dimensions that distinguish mobile applications from stationary applications. The user views one dimension at a time, moves through the dimensions using Previous and Next buttons and can enter the name of an app or feature to receive simple feedback connected to the currently selected dimension. minSdk is the oldest API the app supports, targetSdk is the Android version whose behavior the app targets, and compileSdk is the API level available when compiling. I chose API 24 (minSdk 24) because it was the default selected when I created the project in Android Studio. I kept this value after reviewing what minSdk represents because it provides a reasonable compatibility baseline for this application without requiring me to support much older Android versions (it supports 99.2% at the time of creation). Supporting an older minimum SDK could increase device coverage but may also require more compatibility handling, which didn't seem necessary for the scope of this assignment. The project was generated with targetSdk 37 and compileSdk 37 and I left those values as is

Important Project Files:

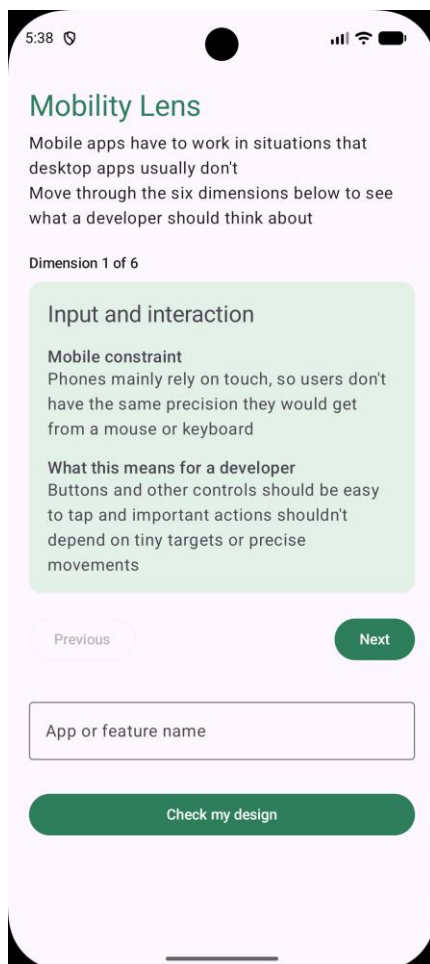
MainActivity.kt is located in app/src/main/java/com/namratha/mobilitylens/, it contains the main activity, the compose screen, the six mobility dimension entries, the navigation logic, the text field state, and the validation behavior. AndroidManifest.xml is located in app/src/main/ and describes important information about the application to the Android operating system. In this project, it declares the application and MainActivity and identifies it as the activity that should launch when the application is opened. The application doesn't request any permissions because none are needed for its functionality. The app level build.gradle.kts configures how the application is built and it contains the namespace, application ID, minSdk, targetSdk, compileSdk, compose configuration, build settings, and dependencies. The version catalog, gradle/libs.versions.toml, keeps plugin and dependency versions and aliases in a central location. strings.xml, located in app/src/main/res/values/, stores UI text such as dimension names, descriptions, recommendations, navigation labels and validation messages instead of placing all text directly in Kotlin

Rotation Observation:

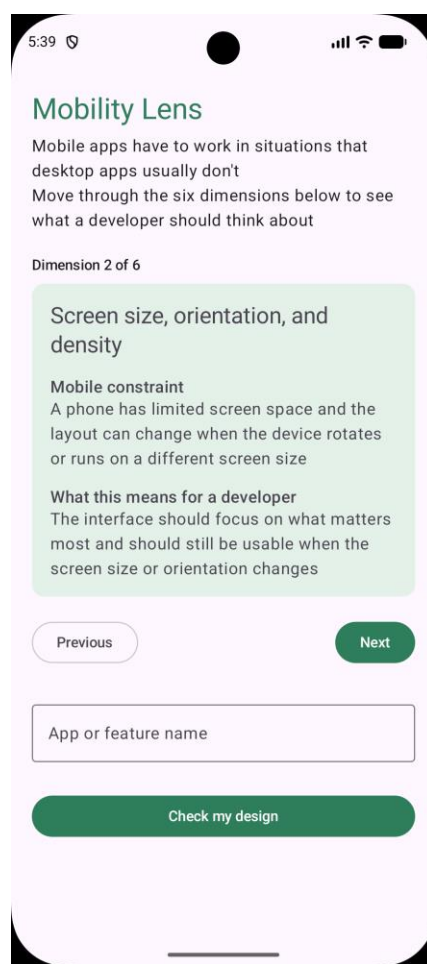
For the rotation test, I navigated to Dimension 2 and entered Security in the App or feature name field. After rotating the Pixel 9 emulator, the application returned to Dimension 1 and the text field became empty. Neither the selected dimension nor the entered text survived the configuration change. These values were stored as Compose state using remember. They survive ordinary recomposition while the current composition remains active but they were lost when the activity and composition were recreated during rotation. Preserving this information can be important because users shouldn't have to re-enter into the text field just because they rotated their phone or due to another configuration change. For this assignment, I left the behavior unchanged, as state restoration will be addressed later.

Mobile Design Decision:

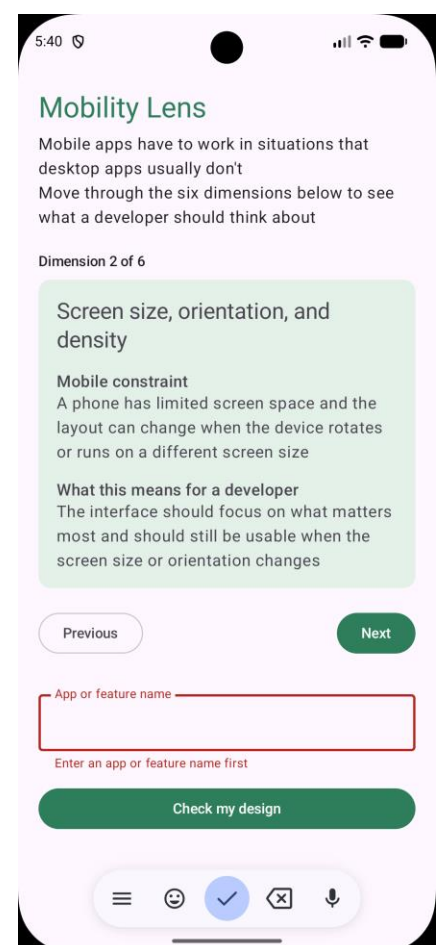
Instead of displaying all mobility dimensions at once, each dimension was shown one at a time. A desktop has enough screen space to show several sections simultaneously but the same layout would become crowded on a phone. The Previous and Next buttons keep the content readable and make the interaction simple on a smaller touch screen. I also made the main Column vertically scrollable using `verticalScroll(rememberScrollState())`, so the content remains accessible when the available vertical space is reduced, like in landscape orientation or on a smaller display. I also considered how the display would look in dark mode (shown in preview)



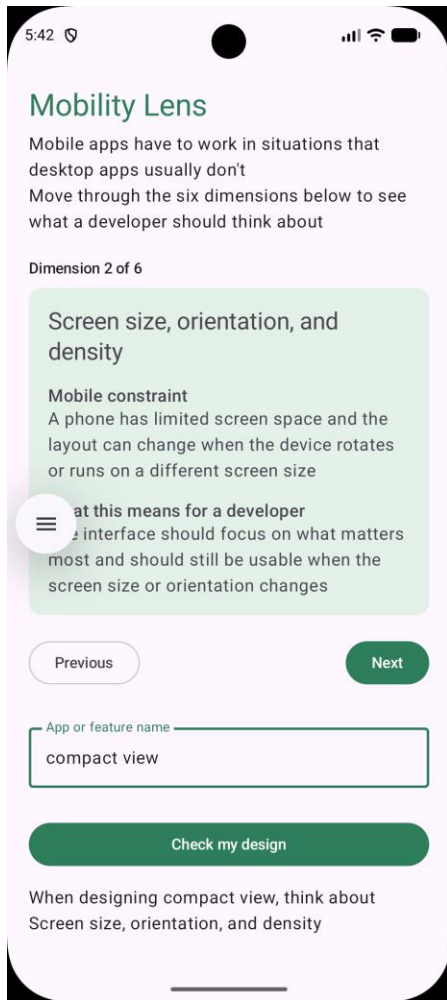
(first dim)



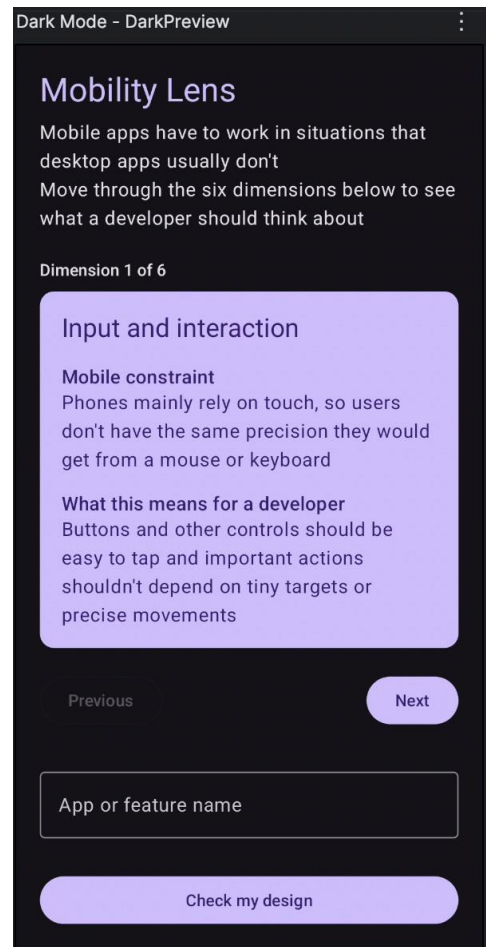
(next dim after user interaction)



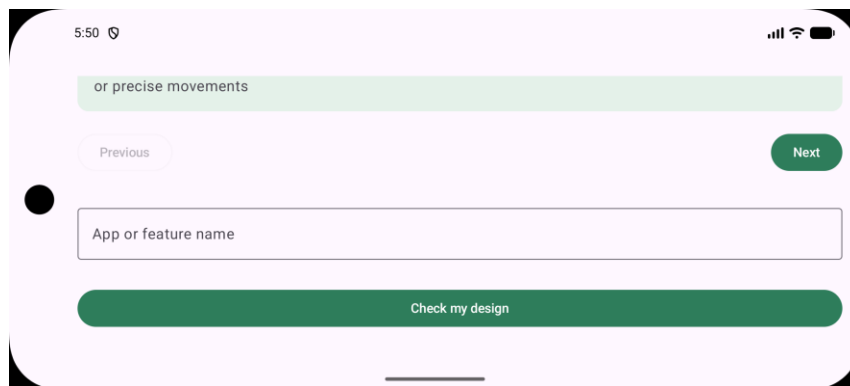
(blank input result)



(valid input result)



(preview of dark mode)



(after rotation)

Collaboration and Generative-AI Disclosure:

I didn't collaborate with anyone for this assignment. I used Gemini 3.6 Flash. I asked for explanations of the assignment requirements and compose concepts, "how do I organize dimensions in a compose app", "explain how remember and compose state work for previous, next and text field," and I asked questions on concepts like, "why do I need preview when I have mainactivity?" Also, I asked questions regarding the Kotlin syntax

Relevant suggestions produced: Gemini suggested representing each mobility dimension with a small data structure, using compose state for the current dimension and user input, disabling Previous/Next at the ends of the list, validating blank input with an OutlinedTextField error state

What I accepted, changed, or rejected: I used the base single screen compose structure, state based navigation, resource organization, and syntax guidance. I reviewed and adjusted the wording and project details to match my application and course requirements rather than treating generated text as the final authority. I didn't add any unnecessary permissions, XML layouts, or extra features that weren't required for this assignment.

Verification: I verified my work by building and running the project in Android Studio on a Pixel 9 API 24 emulator. I manually moved through the mobility dimensions, tested Previous and Next, checked blank and valid text field input, and did the rotation test. I also checked the project configuration and source files and used the running application to determine the rotation result and the smoothness/navigation of the application